Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería Ingeniería en Ciencias y Sistemas.
Laboratorio Compiladores 2



Contacto 3xtrat3rr3str3D Manual Técnico

Septiembre 26 del 2,026

Enmer Oswaldo Sandoval Mazariegos 202031773

Manual Técnico — Compilador Contacto 3xtrat3rr3str3D (Java Swing + ANTLR4)

Requisitos:
JDK 17 o superior
GCC (MinGW-w64) para compilar el código C
4GB de RAM
IDE compatible con JAVA

1) Resumen del proyecto

Aplicación de escritorio en **Java Swing** que compila programas escritos en tres lenguajes: **Y?** (.y), **Zetariano** (.z) y **Pig Latin** (.pig). El archivo **main.pig** es el punto de entrada e importa los archivos de los otros dos lenguajes. Permite:

- Crear carpetas y archivos **.pig**, **.y** y **.z**; abrir proyectos y archivos; guardar; descargar archivos y carpetas.
- Editar el código en un **JTextPane** con **coloreado en tiempo real** para los tres lenguajes, sin librerías.
- **Compilar**: Léxico/Sintaxis (ANTLR4) → AST (visitors) → Semántico (3 pasadas) → Cuartetas → Código de tres direcciones en C → gcc.
- **Mostrar Tabla de Errores** (JTable con *Tipo, Archivo, Línea, Columna, Lexema, Descripción* + doble clic para ir a la línea).
- **Mostrar Reportes**: Tabla de símbolos, Tabla de tipos, Cuartetas y Código de tres direcciones.

Estado: **completo**: análisis léxico, sintáctico y semántico, **cuartetas**, **código C** e **interfaz**.

2) Tecnologías utilizadas

Tecnología	Versión	Uso en el proyecto
Java (JDK)	17 o superior	Lenguaje del proyecto. Se compila con release 17.
Java Swing	Incluido en el JDK	Interfaz gráfica, diseñada con el editor visual de NetBeans (IDERecaf).
ANTLR 4	4.13.2	Genera los lexers y parsers de Y?, Zetariano y Pig Latin. El AST se arma con visitors.
Apache Maven	3.x	Construcción del proyecto. antlr4-maven-plugin genera los analizadores y maven-shade-plugin 3.5.1 arma el JAR con sus dependencias.
Lombok	1.18.46	Genera los getters de las clases (@Getter).
GCC (MinGWw64)	16.1	Compila el archivo programa.c generado por el compilador.
Apache NetBeans	IDE	Entorno de desarrollo y editor de la interfaz.
draw.io	Web	Diagramas de clases.

3) Arquitectura (alto nivel)

3.1 Paquetes principales

- practica.compi2
  - **Main**: Punto de entrada. Sin argumentos abre el IDE; con un archivo compila por consola.
- practica.compi2.vista
  - **IDERecaf**: JFrame principal: árbol de proyecto, editor, consola, menús y botón Compilar.
  - **IdeConsole**: Escribe los mensajes en la consola del IDE.
- practica.compi2.vista.controllers
  - **ProjectController**: Abrir, crear y cerrar proyectos; nuevo archivo; árbol de trabajo.
  - **EditorController**: Abrir, guardar y guardar como; ir a la línea de un error.
  - **CompileController**: Compila, muestra errores y reportes, descarga el código.
  - **FolderController**: Nueva carpeta, descargar archivo y descargar carpeta.
  - **FileTreeBuilder, FileNode**: Construyen el árbol de archivos (JTree).
- practica.compi2.vista.editor
  - **SyntaxHighlighter**: Coloreado en tiempo real sin librerías: recorre los tokens del lexer de ANTLR.
  - **TokenStyles, LexerFactory**: Estilo de cada tipo de token y lexer según la extensión.
  - **FileTemplates**: Plantilla inicial de cada archivo nuevo.
- practica.compi2.vista.reports
  - **ErrorsDialog**: Tabla de errores (doble clic para ir a la línea).
  - **ResultsDialog**: Tabla de símbolos, tabla de tipos, cuartetas y código de tres direcciones.
  - **TableFactory**: Crea las JTable de los reportes.
- practica.compi2.compiler / parsers
  - **Compiler, CompilationResult**: Coordinan las fases y guardan sus resultados.
  - **ProgramLoader**: Carga el main.pig y resuelve los import de forma recursiva.
  - **PigLatinFileParser, YFileParser, ZetarianoFileParser**: Ejecutan el lexer y parser de ANTLR de cada archivo.
- practica.compi2.antlr4 (generados por ANTLR)
  - **YLexer / YParser, ZetarianoLexer / ZetarianoParser, PigLatinLexer / PigLatinParser**: Analizadores léxicos y sintácticos.
  - **YLexerBase**: Maneja la indentación de Y? (INDENT / DEDENT).
- practica.compi2.visitors
  - **YProgramVisitor, ZetarianoClassVisitor, PigLatinProgramVisitor**: Convierten el árbol de ANTLR en nuestro AST.
  - ***StatementVisitor, *ExpressionVisitor, *ControlFlowVisitor, *AtomVisitor, *TypeMapper**: Un visitor por parte del lenguaje.
- practica.compi2.ast
  - **Ast, AstNode, Position**: Raíz del AST, nodo base y posición (línea, columna, lexema).
  - **programs, definitions**: Programas, clases, estructuras, funciones, métodos y constructores.
  - **statements, expressions, operations**: Sentencias, expresiones y operaciones. Cada nodo tiene validate, toIntermediateCode y print.
- practica.compi2.semantic

  SemanticAnalyzer: Hace las 3 pasadas del análisis semántico.
  - **SemanticContext, TypeResolver, CallMatcher, AssignmentChecker**: Ámbitos, tipos, sobrecarga y asignaciones.
  - **symbols: Scope, Frame, GlobalTable, Symbol**: Tabla de símbolos y posiciones en el stack. **types:**
  - **TypeTable, TypeRules, *CompatibilityTable**: Tabla de tipos y reglas de compatibilidad de cada lenguaje.
- practica.compi2.c3d
  - **IntermediateCodeGenerator, IntermediateCodeUtils, IdsCounter**: Generan las cuartetas, temporales y
  - etiquetas. **cuartetas**: Operation3D, Assignation3D, Conditional3D, Goto3D, Call3D, Print3D, etc. **access, helpers,**
  - **natives**: Accesos a stack y heap, ayudantes de generación y las 8 funciones nativas.
  - **CCodeGenerator**: Convierte las cuartetas en programa.c y llama a gcc.
- practica.compi2.errors / console
  - **ErrorManager, CompilationError, SyntaxErrorListener, AntlrMessageTranslator**: Registro de errores y traducción de los mensajes de ANTLR.
  - **ConsoleRunner y printers**: Modo consola: AST, cuartetas, tablas y C3D.

3.2 Flujo de compilación (Compiler)

1. **Carga de archivos** (ProgramLoader)
   - Parte del **main.pig** y sigue cada **import** de forma recursiva. La ruta `modulos.Funciones.y` se busca en la carpeta `modulos/` del proyecto.
   - Cada archivo se analiza con su lexer y parser de ANTLR y un **SyntaxErrorListener** (fases “Léxico” y “Sintáctico”). **AntlrMessageTranslator** traduce los mensajes al español.
   - Los visitors convierten el árbol de ANTLR en el **AST**.
   - Si hay errores léxicos o sintácticos → **Tabla de Errores** + fin. Se reportan los errores de todos los archivos juntos.

   2. **Semántico** (SemanticAnalyzer) **registerTypes** → registra estructuras y clases en la **TypeTable**.
   - **registerMembers** → campos, atributos, funciones, métodos y constructores, con detección de duplicados.
   - **validatePrograms** → cada nodo ejecuta `validate(sc)`: ámbitos (**Scope**), tipos (**TypeRules** y tablas de
   - compatibilidad), sobrecarga (**CallMatcher**) y posición de cada variable en el stack (**Frame**). Si hay errores → **Tabla de Errores** + fin.

   3. **Cuartetas** (IntermediateCodeGenerator)
   Cada nodo ejecuta `toIntermediateCode(u)` y agrega sus cuartetas a **IntermediateCodeUtils**. **IdsCounter** numera temporales
   - (`t0, t1…`) y etiquetas (`et0, et1…`).
   Se agregan las 8 funciones nativas (imprimir cadena, concatenar, convertir entre números y texto, comparar cadenas y leer una
   - línea), también escritas en cuartetas.
4. **Código C** (CCodeGenerator)
   - Cada cuarteta se escribe en C con `toCCode(sb)`. El resultado es **programa.c**, que solo usa asignaciones, operaciones de dos operandos, `if … goto`, `printf` y `getchar`.
   - El IDE guarda programa.c en la carpeta del proyecto y lo compila con **gcc**.

3.3 Modelo de memoria del código generado

- `double stack[1000000]` y `double heap[10000000]`. **P** apunta al inicio del marco actual en el stack y **H** a la siguiente celda libre del heap. **heap[0]** se reserva como **null** y cadena vacía, por eso H inicia en 1.
- Marco de una función: la celda 0 guarda el valor de retorno, la 1 guarda **this** en los métodos, y después van los parámetros y las
- variables locales.
  Llamada: los argumentos se copian después del marco actual, se hace `P = P + tamaño`, se llama la función, se lee `stack[P]` y se
- regresa `P`.
  Cadenas: bytes UTF-8 en el heap terminados en 0. Objetos y estructuras: un bloque de celdas en el heap, una por atributo.
- Arreglos: un bloque en el heap con las dimensiones al inicio y los datos aplanados por filas.
-

4) Diagrama de clases

El diagrama completo está en el archivo **diagrama-clases.drawio**, que se abre en **app.diagrams.net**. Tiene una página por lenguaje, tres páginas con las partes que comparten y una de la interfaz. En el PDF las figuras son vectoriales, así que se pueden ampliar sin perder calidad.

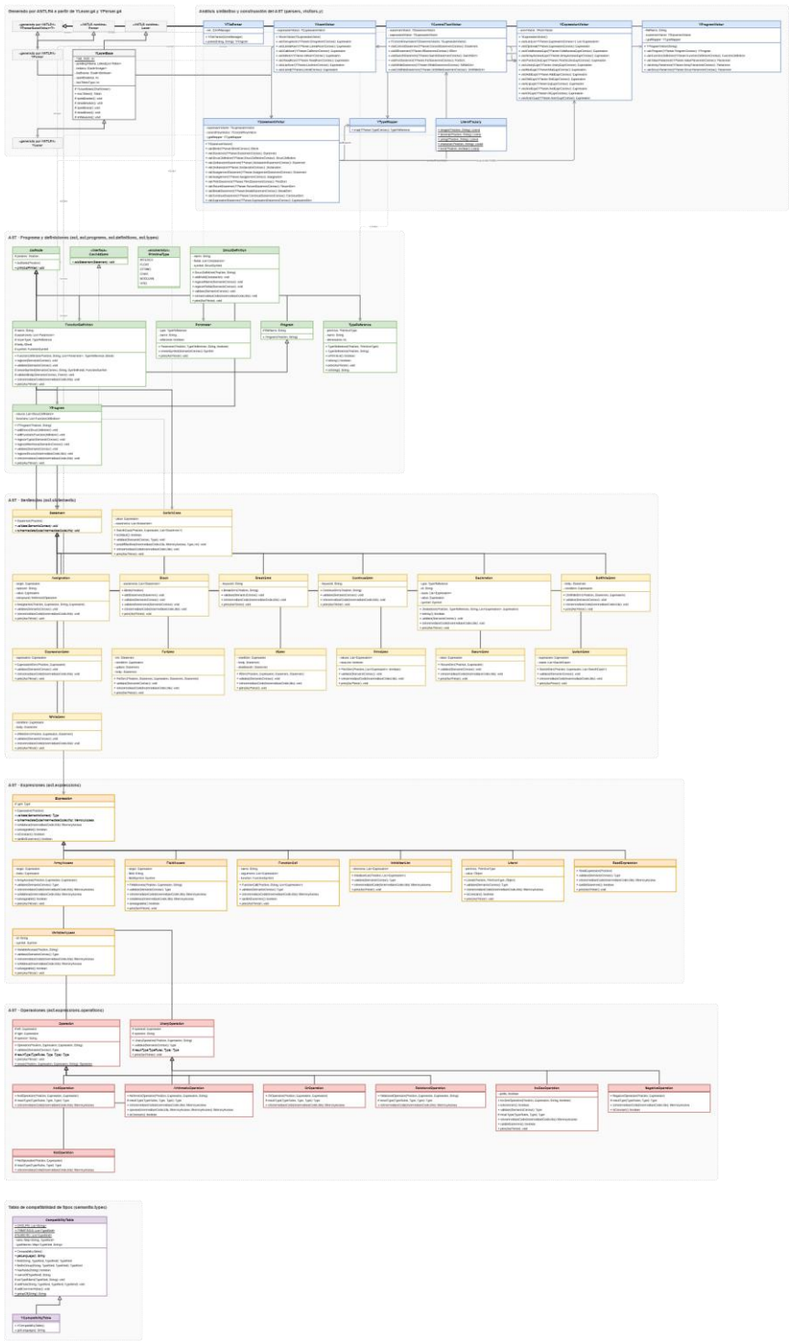


Figura 1. Diagrama de clases: Y?

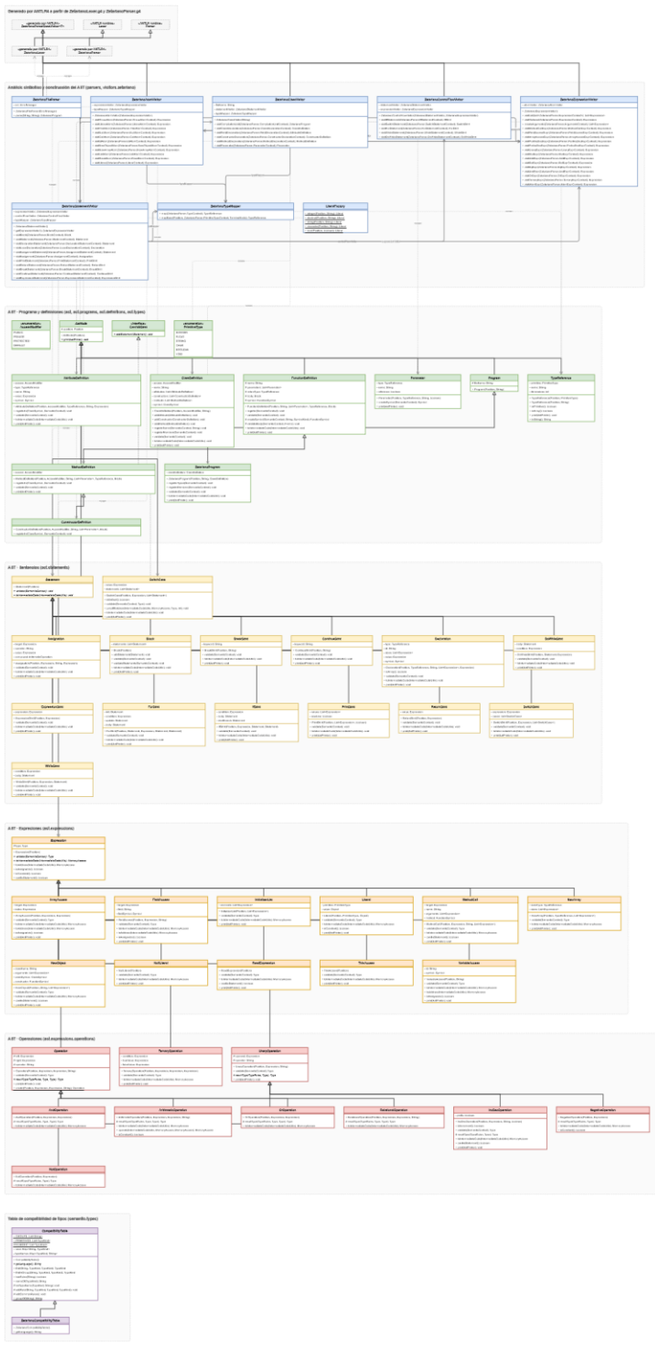


Figura 2. Diagrama de clases: Zetariano

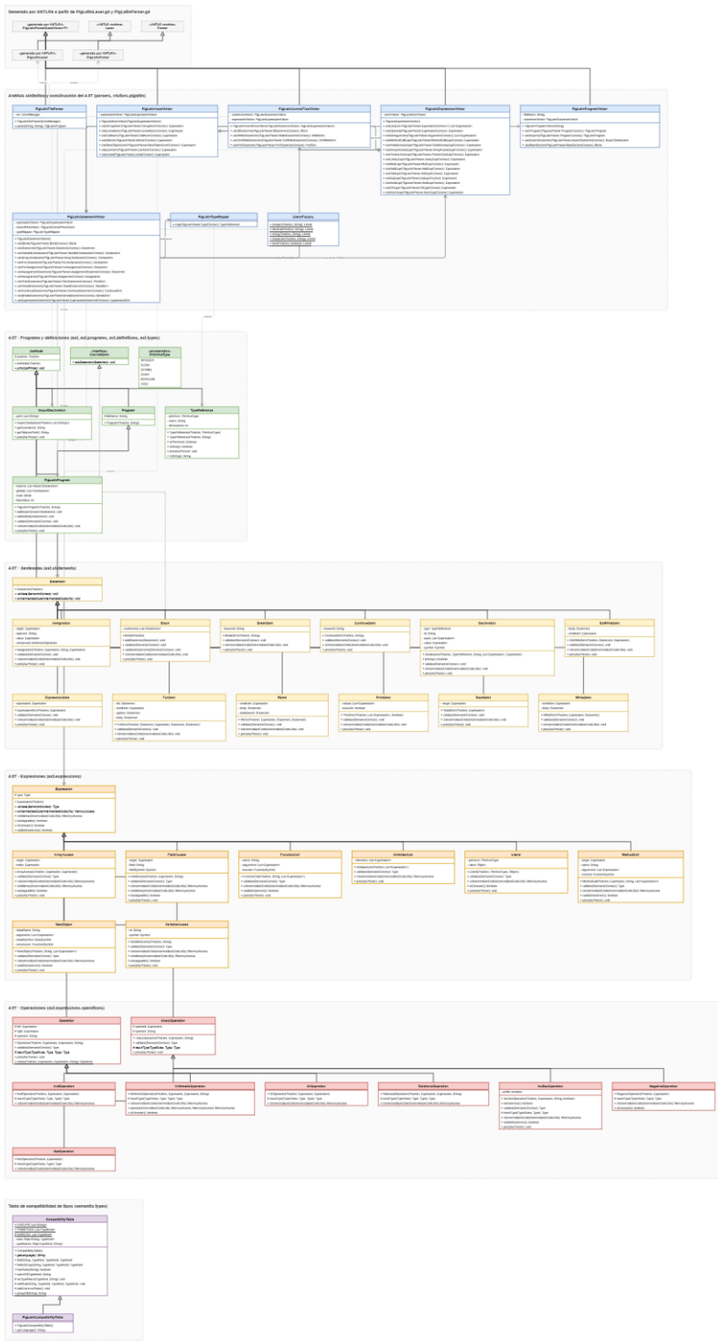


Figura 3. Diagrama de clases: Pig Latin

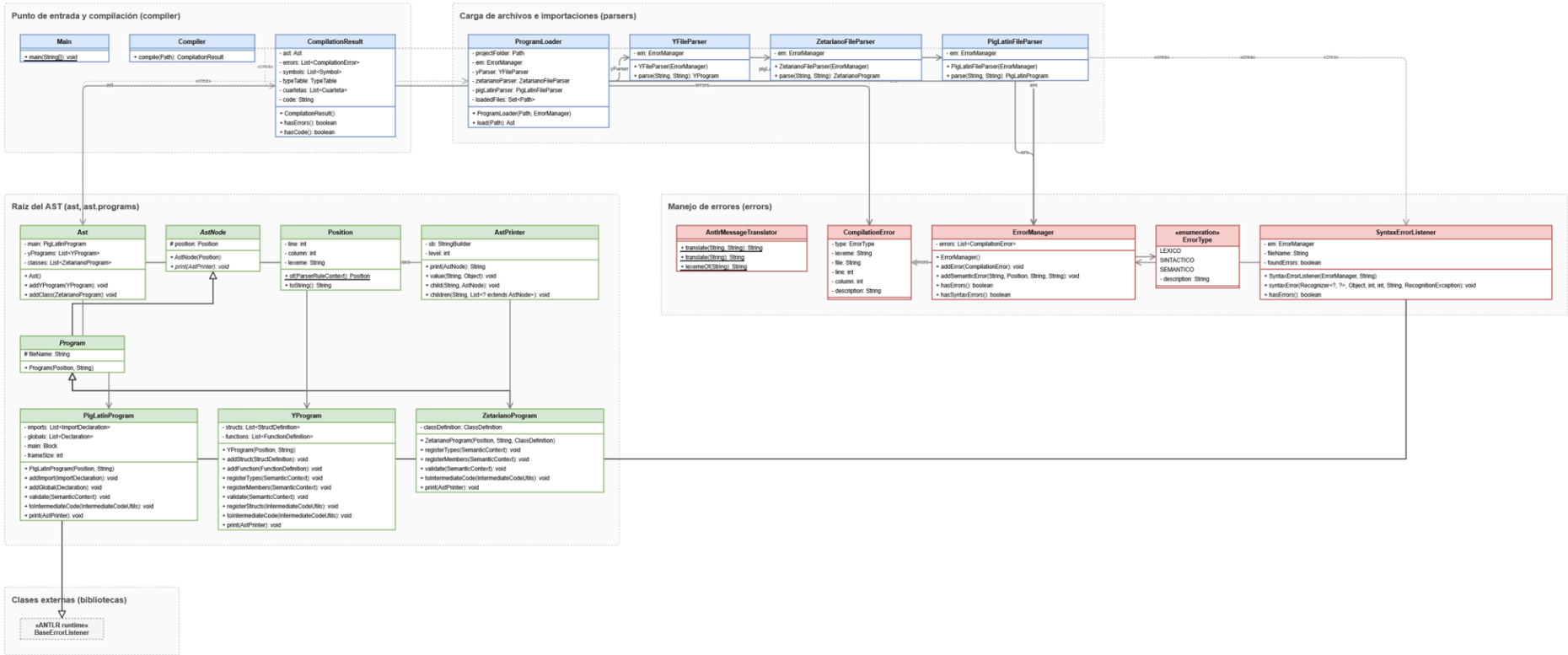


Figura 4. Diagrama de clases: Común · Compilación y errores

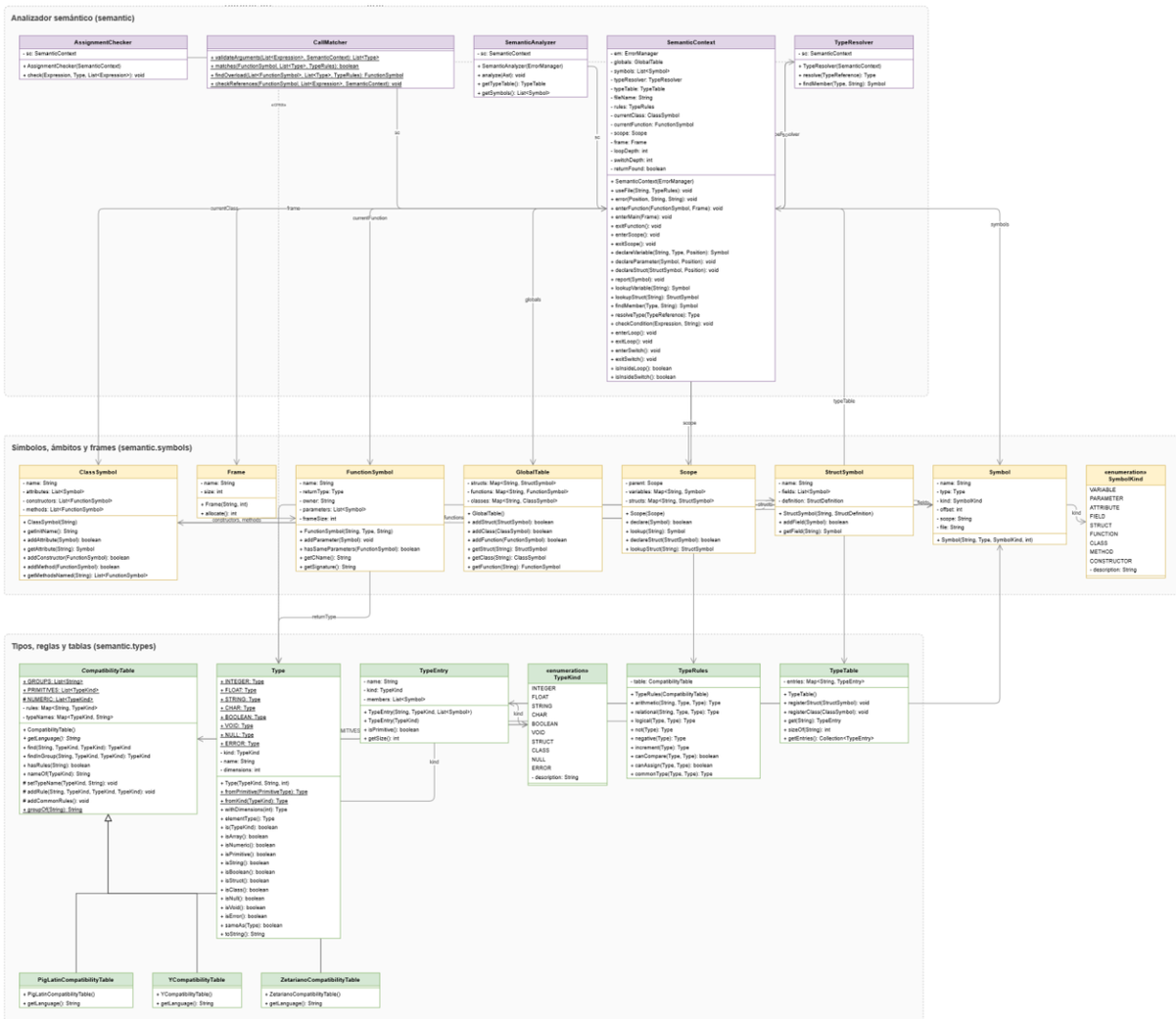


Figura 5. Diagrama de clases: Común · Análisis semántico

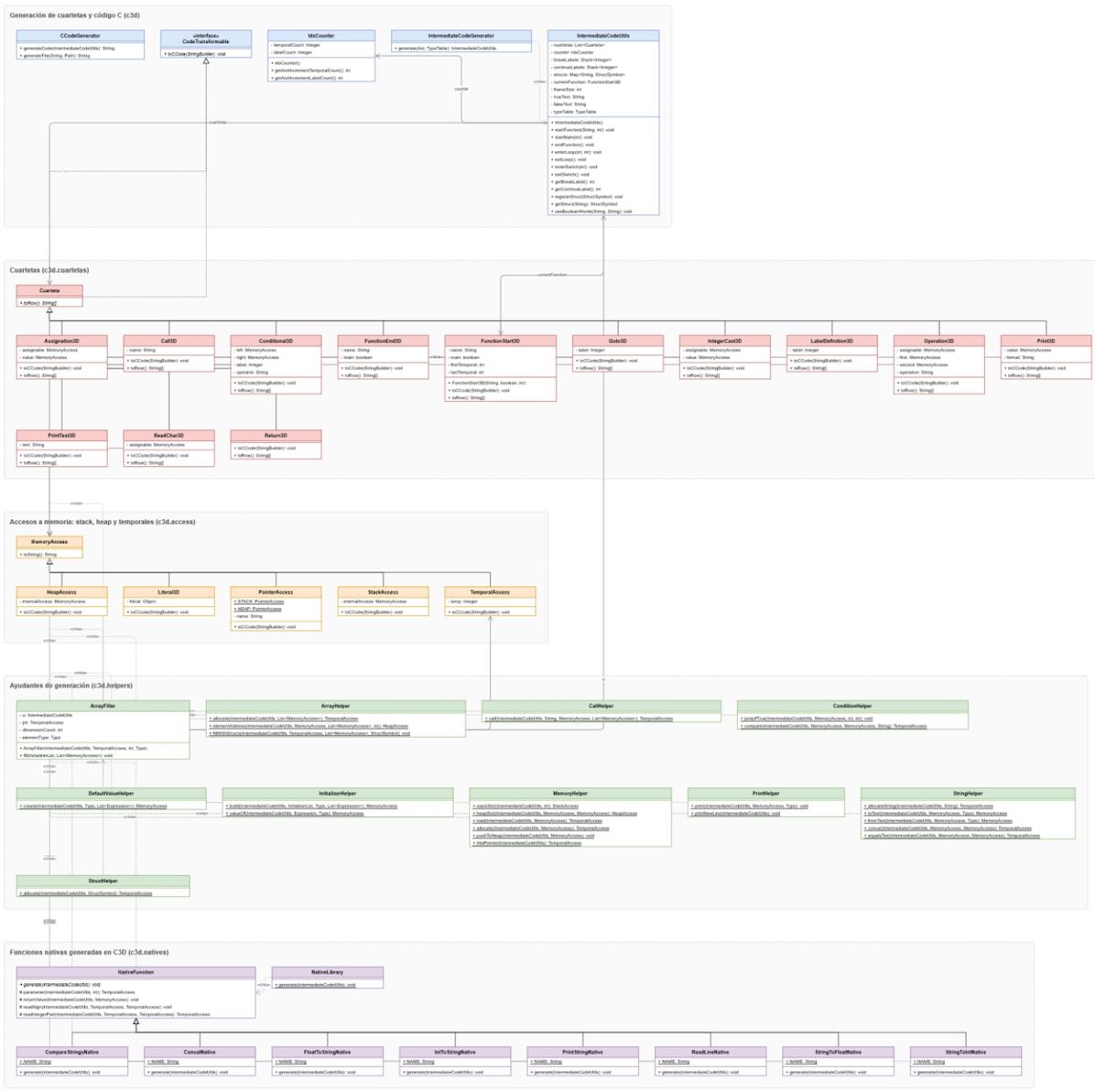


Figura 6. Diagrama de clases: Común · Código de tres direcciones

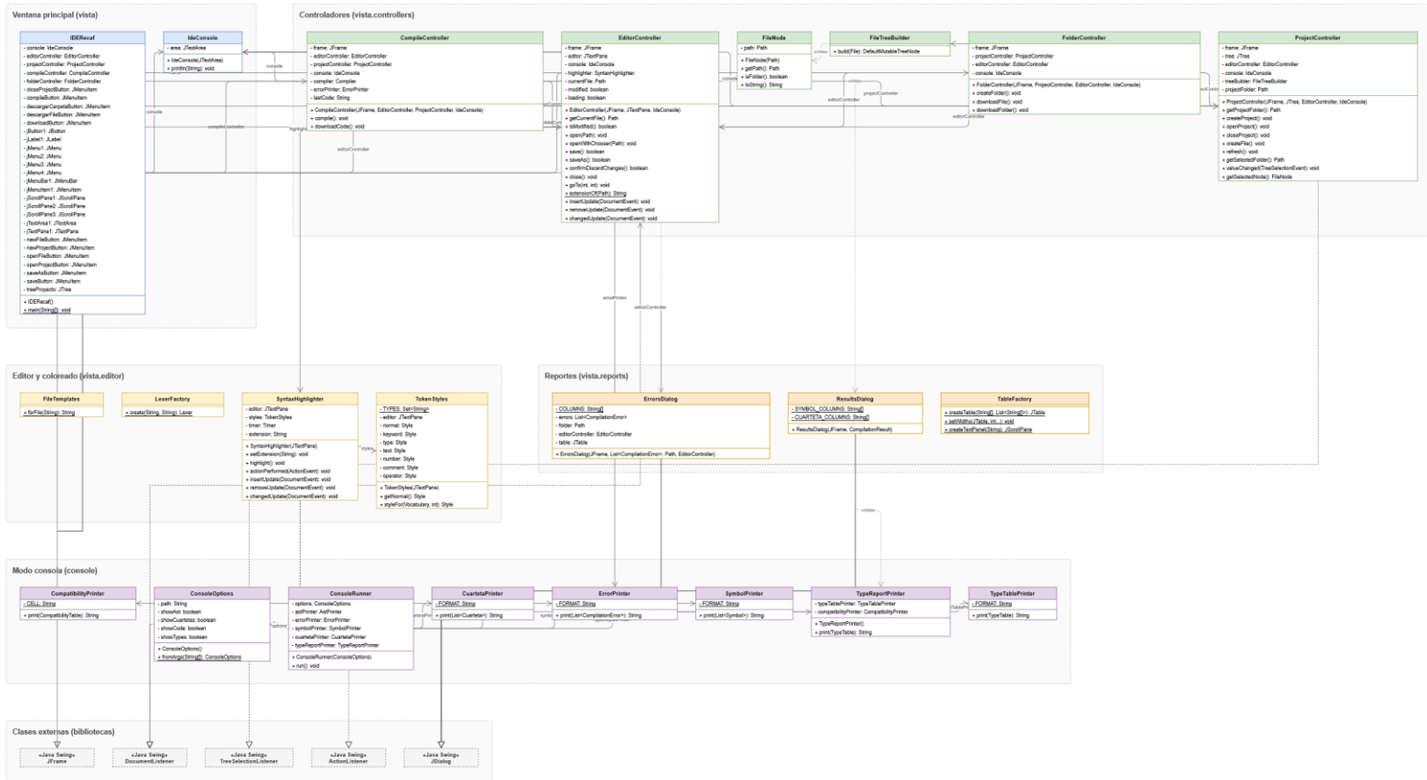


Figura 7. Diagrama de clases: Interfaz gráfica y consola

5) Especificación de los lenguajes

Para cada lenguaje se listan sus palabras reservadas, sus símbolos y su gramática. La gramática es la de ANTLR4 del proyecto, sin las etiquetas de los visitors. Los tres lenguajes comparten los literales:

Literal	Expresión regular	Ejemplo
Entero	`[0-9]+`	`10, 250`
Decimal	`[0-9]+ '.' [0-9]+`	`9.81`
Carácter	`'\'' (escape | carácter) '\''`	`'a', '\n'`
Cadena	`'"' (escape | carácter)* '"'`	`"Hola"`
Identificador	`[a-zA-Z_] [a-zA-Z0-9_]*`	`miVariable, _x1`
Escapes	`\n \r \t \' \" \\`	`"linea\n"`

5.1 Y? (.y)

Palabras reservadas

Palabra	Uso
`%estructuras`	Inicia la sección de estructuras (opcional).
`%funciones`	Inicia la sección de funciones.
`estructura`	Define una estructura con sus campos.
`definir`	Define una función.
`retornar`	Devuelve un valor desde una función.
`si`	Condicional.
`entonces`	Separa la condición del bloque de un si / sino.
`sino`	Condición alterna (si no, si).
`contrario`	Bloque que se ejecuta si ninguna condición se cumplió.
`elegir`	Selección múltiple (switch).
`caso`	Opción de un elegir.
`siempre`	Opción por defecto de un elegir.
`romper`	Sale del ciclo o del elegir.
`continuar`	Salta a la siguiente iteración.
`para`	Ciclo para (for).
`mientras`	Ciclo mientras (while) y condición del hacer.
`hacer`	Ciclo hacer-mientras (do-while).
`imprimir`	Imprime un valor y baja de línea.
`leer`	Lee una línea de la consola.
`verdadero / falso`	Valores booleanos.
`entero`	Tipo entero.
`flotante`	Tipo decimal.
`cadena`	Tipo cadena de texto.
`caracter`	Tipo carácter.
`bool`	Tipo booleano.

Símbolos

Símbolo	Significado
`+`	Suma o concatenación
`-`	Resta o negativo
`*`	Multiplicación
`/`	División
`++ / --`	Incremento y decremento
`== / !=`	Igual y distinto
`< > <= >=`	Comparaciones
`&& / || / !`	Y, O y negación lógica
`=`	Asignación
`( )`	Agrupación, llamadas y condiciones
`[ ]`	Índices y dimensiones de arreglos
`{ }`	Bloques y listas de valores
`,`	Separador de argumentos y valores
`.`	Acceso a campos, atributos y métodos
`->`	Indica el tipo de retorno de una función
`:`	Abre el bloque de una función, estructura, caso o ciclo
`;`	Fin de instrucción (opcional)
`Indentación`	Define los bloques (INDENT / DEDENT)
`// y /* */`	Comentario de línea y de bloque

Gramática

```
program
    : structSection? functionSection EOF
```

```
    ;

structSection
    : SECCION_ESTRUCTURAS NEWLINE structDefinition*
    ;

functionSection
    : SECCION_FUNCIONES NEWLINE functionDefinition*
    ;

structDefinition
    : ESTRUCTURA ID COLON NEWLINE INDENT fieldDefinition+ DEDENT
    ;

fieldDefinition
    : type ID dimensions? SEMICOLON? NEWLINE
    ;

functionDefinition
    : DEFINIR ID LPAREN parameters? RPAREN (ARROW type)? COLON block
    ;
```

```
parameters
    : parameter (COMMA parameter)*
    ;

parameter
    : type ID
    | (LBRACKET RBRACKET)+ type
ID     | LBRACE RBRACE ID ID
    ;

block
    : NEWLINE INDENT statement+ DEDENT
    ;

statement
    : structDefinition
    | simpleStatement SEMICOLON?
NEWLINE     | controlStatement
    ;

simpleStatement
: declaration
    | assignment
    | IMPRIMIR LPAREN expression
RPAREN     | RETORNAR expression?
    | ROMPER
    | CONTINUAR
    | expression
    ;

declaration
    : type ID dimensions? (ASSIGN expression)?
    ;

assignment
    : expression ASSIGN expression
    ;

dimensions
    : (LBRACKET expression RBRACKET)+
    ;

controlStatement
    : ifStatement
    | switchStatement
    | forStatement
    | whileStatement
| doWhileStatement
    ;

ifStatement
    : SI LPAREN expression RPAREN ENTONCES block elseIfClause* elseClause?
    ;

elseIfClause
    : SINO LPAREN expression RPAREN ENTONCES block
    ;

elseClause
    : CONTRARIO block
    ;

switchStatement
    : ELEGIR LPAREN expression RPAREN LBRACE NEWLINE INDENT caseClause* defaultClause? DEDENT RBRACE NEWLINE
    ;

caseClause
    : CASO expression COLON block
    ;

defaultClause
    : SIEMPRE COLON block
    ;

forStatement
    : PARA LPAREN forInit? SEMICOLON expression? SEMICOLON forUpdate? RPAREN COLON block
    ;

forInit
    : declaration
    | assignment
    ;

forUpdate     :
assignment
| expression
    ;

whileStatement
    : MIENTRAS LPAREN expression RPAREN HACER block
    ;

doWhileStatement
    : HACER COLON block MIENTRAS LPAREN expression RPAREN SEMICOLON? NEWLINE
    ;

expression
    : expression DOT ID     | expression
LBRACKET expression RBRACKET
    | expression (INC | DEC)
    | (NOT | MINUS) expression
    | expression (MULT | DIV) expression
    | expression (PLUS | MINUS) expression
| expression (LT | GT | LE | GE) expression
    | expression (EQ | NE) expression
    | expression AND expression
    | expression OR expression
    | atom
;

atom
    : LPAREN expression RPAREN
    | literal
```

```
    | ID LPAREN arguments? RPAREN
    | ID
    | LEER LPAREN RPAREN
    | LBRACE (expression (COMMA expression)*)? RBRACE
```

```
    ;

arguments
    : expression (COMMA expression)*
    ;

literal
    : INT_LIT
    | DECIMAL_LIT
    | STRING_LIT
    | CHAR_LIT
    | VERDADERO
    | FALSO
;

type
    : ENTERO
    | FLOTANTE
    | CADENA
    | CARACTER
    | BOOL
    | ID
    ;
```

5.2 Zetariano (.z)

Palabras reservadas

Palabra	Uso
`public / private / protected`	Modificadores de acceso.
`class`	Define la clase del archivo.
`void`	Método que no devuelve valor.
`new`	Crea un objeto o un arreglo.
`null`	Referencia nula.
`this`	Referencia al objeto actual.
`if / else`	Condicional.
`switch / case / default`	Selección múltiple.
`for`	Ciclo para.
`while`	Ciclo mientras.
`do`	Ciclo hacer-mientras.
`break`	Sale del ciclo o del switch.
`continue`	Salta a la siguiente iteración.
`return`	Devuelve un valor desde un método.
`println`	Imprime un valor y baja de línea.
`print`	Imprime un valor sin bajar de línea.
`readln`	Lee una línea de la consola.
`true / false`	Valores booleanos.
`int`	Tipo entero.
`double`	Tipo decimal.
`char`	Tipo carácter.
`boolean`	Tipo booleano.
`String`	Tipo cadena de texto.

Símbolos

Símbolo	Significado
`+`	Suma o concatenación
`-`	Resta o negativo
`*`	Multiplicación
`/`	División
`++ / --`	Incremento y decremento
`== / !=`	Igual y distinto
`< > <= >=`	Comparaciones
`&& / || / !`	Y, O y negación lógica
`=`	Asignación
`( )`	Agrupación, llamadas y condiciones
`[ ]`	Índices y dimensiones de arreglos
`{ }`	Bloques y listas de valores
`,`	Separador de argumentos y valores
`.`	Acceso a campos, atributos y métodos
`%`	Módulo
`+= -= *= /=`	Asignación compuesta
`? :`	Operador ternario
`:`	Cierra un case o default
`;`	Fin de instrucción
`// y /* */`	Comentario de línea y de bloque

Gramática

```
compilationUnit
    : classDeclaration EOF
    ;

classDeclaration
    : accessModifier? CLASS ID LBRACE classMember* RBRACE
    ;

classMember
    : fieldDeclaration
    | constructorDeclaration
| methodDeclaration
    ;

accessModifier
: PUBLIC
    | PRIVATE
| PROTECTED
    ;

fieldDeclaration
    : accessModifier? type ID (ASSIGN expression)? SEMICOLON
    ;
```

```
constructorDeclaration
    : accessModifier? ID LPAREN parameters? RPAREN block
    ;

methodDeclaration
    : accessModifier? (type | VOID) ID LPAREN parameters? RPAREN block
    ;

parameters
    : parameter (COMMA parameter)*
    ;

parameter
    : type ID
    ;

type
    : (primitiveType | ID) (LBRACKET RBRACKET)*
    ;

primitiveType
: INT
    | DOUBLE
    | CHAR
    | BOOLEAN
    | STRING
;

block
    : LBRACE statement* RBRACE
    ;

statement
    : block
    | simpleStatement SEMICOLON
    | controlStatement
    | SEMICOLON
    ;

simpleStatement
    : localDeclaration
    | assignment
    | (PRINTLN | PRINT) LPAREN expression?
RPAREN     | RETURN expression?
    | BREAK
    | CONTINUE
| expression
    ;

localDeclaration
    : type ID (ASSIGN expression)?
    ;

assignment
    : expression (ASSIGN | ADD_ASSIGN | SUB_ASSIGN | MUL_ASSIGN | DIV_ASSIGN) expression
    ;

controlStatement
    : IF LPAREN expression RPAREN statement (ELSE statement)?
    | SWITCH LPAREN expression RPAREN LBRACE switchSection* RBRACE     | FOR
LPAREN forInit? SEMICOLON expression? SEMICOLON forUpdate? RPAREN statement
    | WHILE LPAREN expression RPAREN statement     | DO
statement WHILE LPAREN expression RPAREN SEMICOLON
    ;

switchSection
    : (CASE expression | DEFAULT) COLON statement*
    ;

forInit
    : localDeclaration
| assignment
    ;

forUpdate     :
assignment
| expression
    ;

expression
    : expression DOT ID LPAREN arguments? RPAREN
    | expression DOT ID     | expression
LBRACKET expression RBRACKET
    | expression (INC | DEC)
    | (INC | DEC) expression
    | (NOT | MINUS) expression
    | expression (MULT | DIV | MOD) expression
    | expression (PLUS | MINUS) expression
| expression (LT | GT | LE | GE) expression
    | expression (EQ | NE) expression
    | expression AND expression
    | expression OR expression     | expression
QUESTION expression COLON expression     | atom
;

atom
    : LPAREN expression RPAREN
    | literal
    | THIS
    | NULL
    | ID LPAREN arguments? RPAREN
    | ID
    | NEW ID LPAREN arguments? RPAREN
    | NEW (primitiveType | ID) (LBRACKET expression
RBRACKET)+     | LBRACE (expression (COMMA expression)*)?
RBRACE
    | READLN LPAREN RPAREN
    ;

arguments
    : expression (COMMA expression)*
    ;
```

```
literal
    : INT_LIT
    | DECIMAL_LIT
```

```
    | STRING_LIT
    | CHAR_LIT
    | TRUE
    | FALSE
    ;
```

5.3 Pig Latin (.pig)

Palabras reservadas

Palabra	Uso
`import`	Importa un archivo .y o .z.
`VARIABILES>`	Inicia la sección de variables globales (opcional).
`MAIOR>`	Inicia la sección principal.
`FINIS`	Termina la sección principal (FINIS;).
`esto`	Declara una variable.
`series`	Declara un arreglo.
`novus`	Crea un objeto de una clase Zetariana.
`si`	Condicional.
`aliter`	Rama sino / sino si de un si.
`finis`	Cierra un bloque si, dum o per.
`dum`	Ciclo mientras y condición del facere.
`facere`	Ciclo hacer-mientras.
`per`	Ciclo para.
`perge`	Salta a la siguiente iteración (continuar).
`interrumpe`	Sale del ciclo (romper).
`verum / falsus`	Valores booleanos.
`numerus`	Tipo entero.
`decimalis`	Tipo decimal.
`textum`	Tipo cadena de texto.
`littera`	Tipo carácter.

Símbolos

Símbolo	Significado
`+`	Suma o concatenación
`-`	Resta o negativo
`*`	Multiplicación
`/`	División
`++ / --`	Incremento y decremento
`== / !=`	Igual y distinto
`< > <= >=`	Comparaciones
`&& / || / !`	Y, O y negación lógica
`=`	Asignación
`( )`	Agrupación, llamadas y condiciones
`[ ]`	Índices y dimensiones de arreglos
`{ }`	Bloques y listas de valores
`,`	Separador de argumentos y valores
`.`	Acceso a campos, atributos y métodos
`>>`	Imprimir (cada >> baja de línea, salvo que el texto ya termine en \n)
`<<`	Leer de consola
`:`	Separa el nombre de la variable de su tipo
`;`	Fin de instrucción (opcional después de >> y <<)
`## ##`	Comentario de bloque
`//`	Comentario de línea

Gramática

```
program
    : importStatement* variablesSection? mainSection EOF
    ;

importStatement
    : IMPORT ID (DOT ID)+ SEMICOLON?
    ;

variablesSection
    : VARIABILES declaration*
    ;
```

```
mainSection
    : MAIOR statement* FINIS_MAIOR SEMICOLON
    ;

declaration
    : ESTO ID COLON type? expression? SEMICOLON     | SERIES ID dimensions COLON type expression? SEMICOLON
    ;

dimensions
    : (LBRACKET expression RBRACKET)+
    ;

block
    : LBRACE statement* RBRACE
    ;

statement     : declaration
    | simpleStatement
| controlStatement
    ;

simpleStatement
    : assignment SEMICOLON     | PRINT expression
(PRINT expression)* SEMICOLON?     | expression?
READ SEMICOLON?
    | PERGE SEMICOLON
    | INTERRUMPE SEMICOLON
| expression SEMICOLON
    ;

assignment
    : expression ASSIGN expression
    ;

controlStatement
    : SI LPAREN expression RPAREN block elseIfClause* elseClause? FINIS SEMICOLON
    | DUM LPAREN expression RPAREN block FINIS SEMICOLON
    | FACERE block DUM LPAREN expression RPAREN SEMICOLON     | PER LPAREN forInit? SEMICOLON
expression? SEMICOLON forUpdate? RPAREN block (FINIS SEMICOLON)?
    ;

elseIfClause
    : ALITER LPAREN expression RPAREN block
    ;

elseClause
    : ALITER block
    ;

forInit
    : ESTO ID COLON type? expression
    | assignment
    ;

forUpdate     :
assignment
| expression
    ;

expression
    : expression DOT ID LPAREN arguments? RPAREN
    | expression DOT ID     | expression
LBRACKET expression RBRACKET
    | expression (INC | DEC)
    | (NOT | MINUS) expression
    | expression (MULT | DIV) expression
    | expression (PLUS | MINUS) expression
| expression (LT | GT | LE | GE) expression
    | expression (EQ | NE) expression
    | expression AND
expression     | expression OR
expression
    | atom
;

atom
    : LPAREN expression RPAREN
    | literal
    | ID LPAREN arguments? RPAREN
    | ID
    | NOVUS ID LPAREN arguments? RPAREN     |
LBRACE (expression (COMMA expression)*)? RBRACE
    ;

arguments
    : expression (COMMA expression)*
    ;

literal
    : INT_LIT
    | DECIMAL_LIT
    | STRING_LIT
    | CHAR_LIT
    | VERUM
    | FALSUS
;

type
    : NUMERUS
    | DECIMALIS
    | TEXTUM
    | LITTERA
    | ID
    ;
```

6) Tablas de compatibilidad de tipos

Cada tabla indica el tipo del resultado de aplicar el operador. La fila es el operando izquierdo y la columna el derecho. En la tabla de **=**, la fila es el tipo de la variable y la columna el tipo del valor que se le asigna. **error** significa que el semántico reporta un error. Por ejemplo, en Pig Latin `textum + numerus` da `textum` (se concatena) y `numerus + decimalis` da `decimalis`.

6.1 Y?

+	entero	flotante	caracter	bool	cadena
entero	entero	flotante	entero	error	cadena
flotante	flotante	flotante	flotante	error	cadena
caracter	entero	flotante	cadena	error	cadena
bool	error	error	error	error	cadena
cadena	cadena	cadena	cadena	cadena	cadena

- * /	entero	flotante	caracter	bool	cadena
entero	entero	flotante	entero	error	error
flotante	flotante	flotante	flotante	error	error
caracter	entero	flotante	entero	error	error
bool	error	error	error	error	error
cadena	error	error	error	error	error

< > <= >=	entero	flotante	caracter	bool	cadena
entero	bool	bool	bool	error	error
flotante	bool	bool	bool	error	error
caracter	bool	bool	bool	error	error
bool	error	error	error	error	error
cadena	error	error	error	error	error

== !=	entero	flotante	caracter	bool	cadena
entero	bool	bool	bool	error	error
flotante	bool	bool	bool	error	error
caracter	bool	bool	bool	error	error
bool	error	error	error	bool	error
cadena	error	error	error	error	bool

&& ||	entero	flotante	caracter	bool	cadena
entero	error	error	error	error	error
flotante	error	error	error	error	error
caracter	error	error	error	error	error
bool	error	error	error	bool	error
cadena	error	error	error	error	error

=	entero	flotante	caracter	bool	cadena
entero	permitido	error	permitido	error	error
flotante	permitido	permitido	permitido	error	error
caracter	error	error	permitido	error	error
bool	error	error	error	permitido	error
cadena	error	error	error	error	permitido

6.2 Zetariano

+	int	double	char	boolean	String
int	int	double	int	error	String
double	double	double	double	error	String
char	int	double	int	error	String
boolean	error	error	error	error	String
String	String	String	String	String	String

%	int	double	char	boolean	String
int	int	error	int	error	error
double	error	error	error	error	error
char	int	error	int	error	error
boolean	error	error	error	error	error
String	error	error	error	error	error

== !=	int	double	char	boolean	String
int	boolean	boolean	boolean	error	error
double	boolean	boolean	boolean	error	error
char	boolean	boolean	boolean	error	error
boolean	error	error	error	boolean	error
String	error	error	error	error	boolean

=	int	double	char	boolean	String
int	permitido	error	permitido	error	error
double	permitido	permitido	permitido	error	error
char	error	error	permitido	error	error
boolean	error	error	error	permitido	error
String	error	error	error	error	permitido

- * /	int	double	char	boolean	String
int	int	double	int	error	error
double	double	double	double	error	error
char	int	double	int	error	error
boolean	error	error	error	error	error
String	error	error	error	error	error

< > <= >=	int	double	char	boolean	String
int	boolean	boolean	boolean	error	error
double	boolean	boolean	boolean	error	error
char	boolean	boolean	boolean	error	error
boolean	error	error	error	error	error
String	error	error	error	error	error

&& ||	int	double	char	boolean	String
int	error	error	error	error	error
double	error	error	error	error	error
char	error	error	error	error	error
boolean	error	error	error	boolean	error
String	error	error	error	error	error

6.3 Pig Latin

+	numerus	decimalis	littera	verum/falsus	textum
numerus	numerus	decimalis	numerus	error	textum
decimalis	decimalis	decimalis	decimalis	error	textum
littera	numerus	decimalis	textum	error	textum
verum/falsus	error	error	error	error	textum
textum	textum	textum	textum	textum	textum

- * /	numerus	decimalis	littera	verum/falsus	textum
numerus	numerus	decimalis	numerus	error	error
decimalis	decimalis	decimalis	decimalis	error	error
littera	numerus	decimalis	numerus	error	error
verum/falsus	error	error	error	error	error
textum	error	error	error	error	error

< > <= >=	numerus	decimalis	littera	verum/falsus	textum
numerus	verum/falsus	verum/falsus	verum/falsus	error	error
decimalis	verum/falsus	verum/falsus	verum/falsus	error	error
littera	verum/falsus	verum/falsus	verum/falsus	error	error
verum/falsus	error	error	error	error	error
textum	error	error	error	error	error

== !=	numerus	decimalis	littera	verum/falsus	textum
numerus	verum/falsus	verum/falsus	verum/falsus	error	error
decimalis	verum/falsus	verum/falsus	verum/falsus	error	error
littera	verum/falsus	verum/falsus	verum/falsus	error	error
verum/falsus	error	error	error	verum/falsus	error
textum	error	error	error	error	verum/falsus

&& ||	numerus	decimalis	littera	verum/falsus	textum
numerus	error	error	error	error	error
decimalis	error	error	error	error	error
littera	error	error	error	error	error
verum/falsus	error	error	error	verum/falsus	error
textum	error	error	error	error	error

=	numerus	decimalis	littera	verum/falsus	textum
numerus	permitido	error	permitido	error	error
decimalis	permitido	permitido	permitido	error	error
littera	error	error	permitido	error	error
verum/falsus	error	error	error	permitido	error
textum	error	error	error	error	permitido

7) Manejo de errores

- **Léxicos**: caracteres que no pertenecen al lenguaje. El lexer los reporta y sigue.
- **Sintácticos**: estructura incorrecta. ANTLR se recupera y sigue, así que se reportan varios errores de una vez. Los mensajes se traducen, por ejemplo “Falta ';'” se coloca después del token anterior.
- **Semánticos**: variables o funciones no declaradas, tipos incompatibles, argumentos que no coinciden, métodos inexistentes, imports que no existen, nombre de clase distinto al del archivo, etc.
- Cada error guarda **tipo, archivo, línea, columna, lexema y descripción**. El IDE los muestra en la consola y en la Tabla de Errores; con doble clic se abre el archivo en la línea del error.
- Si hay errores léxicos o sintácticos no se ejecuta el semántico. Si hay errores semánticos no se genera código.

8) Compilación y ejecución

- Construir el proyecto: `mvn clean package`. ANTLR genera los analizadores y se crea `target/Proyecto_Fase1.jar` con todas sus dependencias.
- Abrir el IDE: `java -jar target/Proyecto_Fase1.jar` (o ejecutar el proyecto desde NetBeans).
- Compilar por consola: `java -jar target/Proyecto_Fase1.jar ruta/main.pig [--ast] [--cuartetas] [--c3d] [--tipos].`
- Compilar el código generado a mano: `gcc programa.c -o programa`.